Comprehensive Exam Preparation Guide: Word Indexer Utility

Part 1: 25 Predicted Exam Questions & Solutions

Category 1: Input Parsing & Tokenization

Q1: Modify the indexing logic to make word indexing case-insensitive (e.g., merge "Apple" and "apple").

Python

```
# In indexer.py: Update parse_and_index()

def parse_and_index(input_file, index_struct, case_sensitive=False):

    pattern = re.compile(r'[^ \t\n\r\f\v\.\,\:\;\ ]+')

    with open(input_file, 'r', encoding='ascii', errors='ignore') as f:

        for line_num, line in enumerate(f, start=1):

            for word in pattern.findall(line):

                if not case_sensitive:

                    word = word.lower()

                index_struct.insert(word, line_num)
```

Q2: Filter out common stop-words using a hardcoded set or external file.

Python

```
# In indexer.py: Add stop-word filtering to parse_and_index()

STOP_WORDS = {"the", "a", "an", "and", "or", "in", "on", "at", "to", "of", "is", "it"}


def parse_and_index_with_stopwords(input_file, index_struct, stop_words_file=None):

    stopwords = set(STOP_WORDS)

    if stop_words_file and os.path.exists(stop_words_file):

        with open(stop_words_file, 'r', encoding='ascii', errors='ignore') as sf:

            stopwords.update(sf.read().split())
```

```
    pattern = re.compile(r'[^ \t\n\r\f\v\.\,\:\;\ ]+')
    with open(input_file, 'r', encoding='ascii', errors='ignore') as f:
        for line_num, line in enumerate(f, start=1):
            for word in pattern.findall(line):
                w_check = word.lower()
                if w_check not in stopwords:
                    index_struct.insert(word, line_num)
```

Q3: Filter words based on length constraints ($min_len \leq$ length $\leq max_len$) and pure alphabetic characters.

Python

```
# In indexer.py: Add length and character condition
def parse_and_index_filtered(input_file, index_struct, min_len=3, max_len=15, alpha_only=True):
    pattern = re.compile(r'[^ \t\n\r\f\v\.\,\:\;\ ]+')
    with open(input_file, 'r', encoding='ascii', errors='ignore') as f:
        for line_num, line in enumerate(f, start=1):
            for word in pattern.findall(line):
                if min_len <= len(word) <= max_len:
                    if not alpha_only or word.isalpha():
                        index_struct.insert(word, line_num)
```

Q4: Modify the regular expression to retain intra-word punctuation like hyphens and apostrophes (e.g., "state-of-the-art", "don't").

Python

```
# In indexer.py: Update pattern regex in parse_and_index()
def parse_and_index_custom_tokens(input_file, index_struct):
    # Matches alphanumeric sequences with embedded hyphens or apostrophes
    pattern = re.compile(r"[a-zA-Z0-9]+(?:['\-][a-zA-Z0-9]+)*")
```

```
    with open(input_file, 'r', encoding='ascii', errors='ignore') as f:
        for line_num, line in enumerate(f, start=1):
            for word in pattern.findall(line):
                index_struct.insert(word, line_num)
```

Q5: Modify LineNode and LineQueue to track both Line Number and Word Column/Position on that line.

Python

```
# In indexer.py: Extend LineNode and LineQueue
class PositionNode:
    def __init__(self, line_num, col_num):
        self.line_num = line_num
        self.col_num = col_num
        self.next = None

class PositionLineQueue:
    def __init__(self):
        self.head = self.tail = None

    def enqueue(self, line_num, col_num):
        node = PositionNode(line_num, col_num)
        if not self.head:
            self.head = self.tail = node
        else:
            self.tail.next = node
            self.tail = node
```

```
    def to_list(self):
        res, curr = [], self.head
        while curr:
            res.append(f"L{curr.line_num}:C{curr.col_num}")
            curr = curr.next
        return res
```

Category 2: Core Data Structure Modifications

Q6: Add Autocomplete / Prefix Search functionality (starts_with(prefix)) to TrieIndex.

Python

```
# In indexer.py: Add to TrieIndex class
def starts_with(self, prefix):
    curr = self.root
    for char in prefix:
        if char not in curr.children:
            return []
        curr = curr.children[char]

    results = []
    def dfs(node, path):
        if node.is_word:
            results.append((prefix + path, node.lines.to_list()))
        for char in sorted(node.children.keys()):
            dfs(node.children[char], path + char)

    dfs(curr, "")
```

```
    return results
```

Q7: Implement single-character Wildcard Search (? matching any character) in TrieIndex.

Python

```
# In indexer.py: Add to TrieIndex class
def search_wildcard(self, pattern):
    results = []
    def dfs(node, idx, path):
        if idx == len(pattern):
            if node.is_word:
                results.append((path, node.lines.to_list()))
            return
        ch = pattern[idx]
        if ch == '?':
            for c in sorted(node.children.keys()):
                dfs(node.children[c], idx + 1, path + c)
        elif ch in node.children:
            dfs(node.children[ch], idx + 1, path + ch)

    dfs(self.root, 0, "")
    return results
```

Q8: Delete a word from TrieIndex while unlinking unused internal nodes.

Python

```
# In indexer.py: Add to TrieIndex class
def delete(self, word):
    def _delete(node, word, depth):
```

```
        if not node:
            return False
        if depth == len(word):
            if not node.is_word:
                return False
            node.is_word = False
            return len(node.children) == 0
        char = word[depth]
        if char not in node.children:
            return False
        should_delete = _delete(node.children[char], word, depth + 1)
        if should_delete:
            del node.children[char]
            return len(node.children) == 0 and not node.is_word
        return False


    _delete(self.root, word, 0)
```

Q9: Implement Lexicographical Range Search ([start_word, end_word]) with pruning in BSTIndex.

Python

```
# In indexer.py: Add to BSTIndex class
def search_range(self, start_word, end_word):
    results = []
    def inorder_range(node):
        if not node:
            return
```

```
        if node.word > start_word:
            inorder_range(node.left)
        if start_word <= node.word <= end_word:
            results.append((node.word, node.lines.to_list()))
        if node.word < end_word:
            inorder_range(node.right)

    inorder_range(self.root)
    return results
```

Q10: Delete a word from BSTIndex handling 0, 1, and 2-child cases (in-order successor replacement).

Python

```
# In indexer.py: Add to BSTIndex class
def delete(self, word):
    def _min_value_node(node):
        curr = node
        while curr.left:
            curr = curr.left
        return curr

    def _delete(root, word):
        if not root:
            return root
        if word < root.word:
            root.left = _delete(root.left, word)
        elif word > root.word:
```

```
            root.right = _delete(root.right, word)
        else:
            if not root.left:
                return root.right
            elif not root.right:
                return root.left
            temp = _min_value_node(root.right)
            root.word, root.lines = temp.word, temp.lines
            root.right = _delete(root.right, temp.word)
        return root


    self.root = _delete(self.root, word)
```

Q11: Convert BSTIndex.get_lexicographical_index() to an iterative traversal to prevent recursion stack overflow on deep/skewed trees.

Python

```
# In indexer.py: Replace or add method in BSTIndex class
def get_lexicographical_index_iterative(self):
    result, stack, curr = [], [], self.root
    while stack or curr:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        result.append((curr.word, curr.lines.to_list()))
        curr = curr.right
    return result
```

Q12: Compress consecutive line numbers into formatted ranges (e.g., 1, 2, 3, 5, 6 → "1-3, 5-6").

Python

```
# In indexer.py: Add helper function

def compress_line_numbers(line_list):
    if not line_list:
        return ""
    nums = sorted(list(map(int, line_list)))
    ranges = []
    start = prev = nums[0]


    for n in nums[1:]:
        if n == prev + 1:
            prev = n
        else:
            ranges.append(f"{start}-{prev}" if start != prev else f"{start}")
            start = prev = n
    ranges.append(f"{start}-{prev}" if start != prev else f"{start}")
    return ", ".join(ranges)
```

Category 3: Multi-Word Search Queries

Q13: Implement an AND Query (Line Intersection) returning lines where BOTH word1 AND word2 occur.

Python

```
# In indexer.py: Helper function for Line Intersect

def query_and(index_struct, word1, word2):
    w1_found = index_struct.search(word1)
    w2_found = index_struct.search(word2)
```

```
    if not w1_found or not w2_found:
        return []

    # Get line lists from structural entries
    index_dict = dict(index_struct.get_lexicographical_index())
    l1 = set(map(int, index_dict.get(word1, [])))
    l2 = set(map(int, index_dict.get(word2, [])))

    return sorted(list(l1.intersection(l2)))
```

Q14: Implement an OR Query (Line Union) returning lines where EITHER word1 OR word2 occurs.

Python

```
# In indexer.py: Helper function for Line Union
def query_or(index_struct, word1, word2):
    index_dict = dict(index_struct.get_lexicographical_index())
    l1 = set(map(int, index_dict.get(word1, [])))
    l2 = set(map(int, index_dict.get(word2, [])))

    return sorted(list(l1.union(l2)))
```

Q15: Implement a NOT Query (Line Difference) returning lines where word1 occurs, but NOT word2.

Python

```
# In indexer.py: Helper function for Line Difference
def query_not(index_struct, word1, word2):
    index_dict = dict(index_struct.get_lexicographical_index())
    l1 = set(map(int, index_dict.get(word1, [])))
    l2 = set(map(int, index_dict.get(word2, [])))
```

```
    return sorted(list(l1.difference(l2)))
```

Category 4: Custom Output Formatting & Sorting

Q16: Output the index in Reverse Lexicographical Order ($Z \to A$).

Python

```
# Modify get_lexicographical_index() or sort result
def write_output_reverse(index_data, output_file):
    sorted_reverse = sorted(index_data, key=lambda x: x[0], reverse=True)
    with open(output_file, 'w', encoding='utf-8') as f:
        for word, lines in sorted_reverse:
            f.write(f"{word}: {', '.join(lines)}\n")
```

Q17: Sort output primary by Total Occurrences / Frequency (Descending), secondary by Lexicographical Order (Ascending).

Python

```
def write_output_by_frequency(index_data, output_file):
    # Sort key: (-frequency, word)
    sorted_freq = sorted(index_data, key=lambda x: (-len(x[1]), x[0]))
    with open(output_file, 'w', encoding='utf-8') as f:
        for word, lines in sorted_freq:
            f.write(f"{word} (Freq: {len(lines)}): {', '.join(lines)}\n")
```

Q18: Sort output primary by Word Length (Shortest to Longest), secondary Lexicographically.

Python

```
def write_output_by_length(index_data, output_file):
    sorted_len = sorted(index_data, key=lambda x: (len(x[0]), x[0]))
    with open(output_file, 'w', encoding='utf-8') as f:
        for word, lines in sorted_len:
```

```
        f.write(f"{word} (Length: {len(word)}): {', '.join(lines)}\n")
```

Q19: Filter output to include only words appearing at least K times across the document.

Python

```
def write_output_threshold(index_data, output_file, min_frequency=5):
    with open(output_file, 'w', encoding='utf-8') as f:
        for word, lines in index_data:
            if len(lines) >= min_frequency:
                f.write(f"{word} (Count: {len(lines)}): {', '.join(lines)}\n")
```

Q20: Export Top-K Most Frequent Words with occurrence counts formatted cleanly.

Python

```
def write_top_k_words(index_data, output_file, top_k=50):
    sorted_freq = sorted(index_data, key=lambda x: -len(x[1]))[:top_k]
    with open(output_file, 'w', encoding='utf-8') as f:
        f.write(f"=== TOP {top_k} MOST FREQUENT WORDS ===\n")
        for rank, (word, lines) in enumerate(sorted_freq, start=1):
            f.write(f"{rank:2d}. {word:<15} Count: {len(lines):<5} Lines: {', '.join(lines[:10])}...\n")
```

Category 5: Diagnostics, Memory Benchmarking & Matplotlib

Q21: Add diagnostic methods to BSTIndex to compute Tree Height, Total Nodes, and Leaf Node Count.

Python

```
# In indexer.py: Add to BSTIndex class
def get_diagnostics(self):
    def _stats(node):
        if not node:
            return 0, 0, 0  # height, total_nodes, leaf_nodes
        lh, lt, ll = _stats(node.left)
```

```
        rh, rt, rl = _stats(node.right)

        height = 1 + max(lh, rh)
        total = 1 + lt + rt
        leaves = (1 if not node.left and not node.right else 0) + ll + rl
        return height, total, leaves

    h, t, l = _stats(self.root)
    return {"height": h, "total_nodes": t, "leaf_nodes": l}
```

Q22: Add diagnostic methods to TrieIndex to count Total Internal Trie Nodes vs. Distinct Word Nodes.

Python

```
# In indexer.py: Add to TrieIndex class
def get_diagnostics(self):
    def _count(node):
        total_nodes = 1
        word_nodes = 1 if node.is_word else 0
        for child in node.children.values():
            tn, wn = _count(child)
            total_nodes += tn
            word_nodes += wn
        return total_nodes, word_nodes

    total, words = _count(self.root)
    return {"total_trie_nodes": total, "distinct_words": words}
```

Q23: Integrate Peak Memory Profiling using tracemalloc in benchme.py.

Python

```
# In benchme.py: Add memory profiling wrapper
import tracemalloc

def profile_structure_memory(parse_func, filepath, struct_class):
    tracemalloc.start()
    struct = struct_class()
    parse_func(filepath, struct)
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    peak_mb = peak / (1024 * 1024)
    return struct, peak_mb
```

Q24: Demonstrate BST Worst-Case Performance Degradation ($O(V^2)$) on Pre-Sorted Word Input vs Random Input.

Python

```
# Test BST creation with sorted words
def benchmark_bst_sorted_impact(words_list):
    bst_random = BSTIndex()
    t1 = time.perf_counter()
    for w in words_list:
        bst_random.insert(w, 1)
    time_random = time.perf_counter() - t1

    sorted_words = sorted(words_list)
    bst_sorted = BSTIndex()
    t1 = time.perf_counter()
```

```
    for w in sorted_words:
        bst_sorted.insert(w, 1)
    time_sorted = time.perf_counter() - t1

    print(f"Random Insertion Time: {time_random:.4f}s")
    print(f"Sorted Insertion Time (Skewed Tree): {time_sorted:.4f}s")
```

Q25: Convert benchme.py performance line plots into a Grouped Bar Chart using Matplotlib.

Python

```
# In benchme.py: Replace line plot section with grouped bar chart
import numpy as np

def plot_grouped_bars(dataset_counts, trie_times, bst_times, dict_times, title, filename):
    x = np.arange(len(dataset_counts))
    width = 0.25

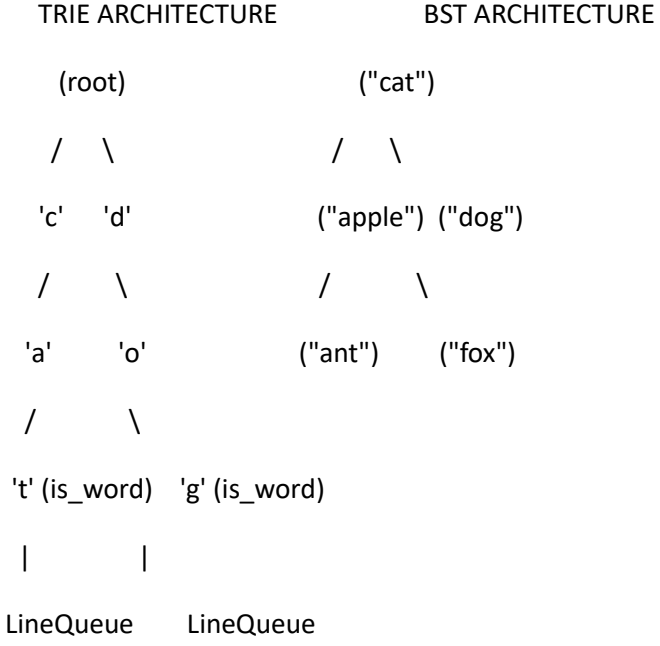
    plt.figure(figsize=(10, 6))
    plt.bar(x - width, trie_times, width, label='Trie')
    plt.bar(x, bst_times, width, label='BST')
    plt.bar(x + width, dict_times, width, label='Dict')

    plt.xlabel('Distinct Words')
    plt.ylabel('Seconds')
    plt.title(title)
    plt.xticks(x, dataset_counts)
    plt.legend()
```

```
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.tight_layout()
    plt.savefig(filename)
    plt.close()
```

Part 2: Essential Conceptual Cheat Sheet & Quick Notes

1. Data Structure Architectures



- **Trie (Prefix Tree)**:
    - **Mechanism**: Nodes store characters in dynamic Python dictionaries (self.children). Words share prefix paths.
    - **Time Complexity**: Insertion/Search costs $O(L)$, where L is word length. Independent of the total number of words V.
    - **Memory Drawback**: High Python dictionary overhead per character node.
- **Binary Search Tree (BST)**:
    - **Mechanism**: Dynamic tree ordering keys by standard string comparison (<, >).
    - **Time Complexity**: Average lookup/insert is $O(L \log V)$. Worst-case is $O(L \cdot V)$ if words arrive pre-sorted (forming a single long linked list).

    - **In-Order Traversal**: Left Subtree → Root → Right Subtree yields lexicographically sorted output automatically in $O(V)$ time.

- **Python Dictionary (Hash Map)**:
    - **Mechanism**: CPython hash table mapping string keys to LineQueue references.
    - **Time Complexity**: Average insertion and lookup in $O(L)$ time due to efficient hashing.
    - **Sorting Requirement**: Keys are unordered; requires explicit sorting via sorted(store.keys()) incurring $O(V \log V)$ sort time for lexicographical output.